Hybrid Navigation System for Autonomous Vehicle using Extended RTAB-Map and Reduced FAST-SCNN

Azzam Wildan Maulana
Department of Electrical Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
6022241101@student.its.ac.id

Rudy Dikairono
Department of Electrical Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
rudydikairono@ee.its.ac.id

Ahmad Zaini
Department of Electrical Engineering
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
zaini@its.ac.id

Moh Ismarintan Zazuli
Department of
Electrical Automation Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
ismarintan@its.ac.id

Muhtadin
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
muhtadin@ee.its.ac.id

Mauridhi Hery Purnomo
Department of Electrical Engineering
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
hery@ee.its.ac.id

***Abstract*—The navigation system is a crucial component of an autonomous vehicle, especially in industrial environments where safety and efficiency are paramount. It is typically divided into two main parts: the global navigation system and the local navigation system. Most existing indoor global navigation systems rely on visual SLAM, which suffers from low update rates (around 1 Hz) and strong dependence on lighting conditions and visual features. To address those limitations, this research proposes an Extended RTAB-Map that introduces a skip connection between wheels and IMU odometry and pose fusion stage. For the local navigation system, many system rely on LiDAR or image segmentation to determine the motion control. There are a lot of lightweight segmentation based on Machine Learning such as Fast-SCNN and D-Fast-SCNN. However, they still have computationally heavy for CPU-only operation. We developed a Reduced Fast-SCNN by reducing the number of convolutional channels and utilizing average pooling. The proposed method achieves 7.63 ms inference time on the same hardware. By combining the Extended RTAB-Map and Reduced Fast-SCNN with the same hardware specification, our method yields the update rate up to 50 Hz, the error pose less than 5 cm relative to the ground truth, and the inference time average for lane detection around 7.63 ms.**

***Index Terms*—navigation system, Fast-SCNN, RTAB-Map SLAM, lane detection, autonomous vehicle**

I. Introduction

As reported in [1], night-shift workers are particularly susceptible to micro-sleep episodes lasting one or two seconds, which can result in serious work accidents. Some companies utilize autonomous vehicles in their system to assist the operators to operate the system ensuring continuity and safety of the material handling. Nowadays, autonomous vehicles are increasingly vital in industrial logistics, offering improvements in safety, efficiency, and operational precision. The problem in autonomous vehicle navigation system are misaligned pose estimation, the update rate of pose estimation, and the inference time of lane detection [2].

An autonomous navigation system generally consists of two main components: the global navigation system for estimating pose of vehicle, and the local navigation system for controlling vehicle motion inside vehicle lookahead distance. The global system requires both accuracy and high update rates to maintain control stability. The RTAB-Map provides a limited update rate (around 1 Hz) and heavily dependent on visual odometry, which suffers under lighting variations or occlusions [3] [4]. We proposed an Extended RTAB-Map by utilizing RTAB-Map and introducing a skip connection from wheels and IMU odometry to pose fusion to improve the frequency of the update rate.

Lane detection as part of local navigation system is essential for maintaining motion control safely. Deep learning-based semantic segmentation models like Fast-SCNN [5] are efficient but still computationally heavy for CPU-only systems. Although D-Fast-SCNN [6] improves the accuracy, however, the inference time is also increase. We proposed a *Reduced Fast-SCNN* by reducing the number of convolutional channels and utilizing average pooling to improve the update rate of lane detection to meet real-time industrial demands. The Extended RTAB-Map and Reduced Fast-SCNN are our contributions in this paper to improve the update rate of pose estimation, the accuracy of pose estimation, and the inference time of lane detection.

II. METHODOLOGY

The output segmentation provides a confidence level used to adjust vehicle speed dynamically. By integrating the Extended RTAB-Map and Reduced Fast-SCNN, this research delivers a high-frequency, CPU-efficient, and centimeter-accurate navigation system suitable for industrial autonomous vehicles.

The methodology of this research is divided into three main parts. The first part discusses the architecture of the Extended RTAB-Map. The second part presents the network design of the Reduced Fast-SCNN. The third part explains the integration of the Extended RTAB-Map and Reduced Fast-SCNN into a unified navigation system for autonomous industrial vehicles.

A. Extended RTAB-Map

The Extended RTAB-Map is a framework to improve the update rate based on RTAB-Map framework for global navigation system. We extend the RTAB-Map framework by modified several parts of RTAB-Map architecture as shown in Fig. 1.

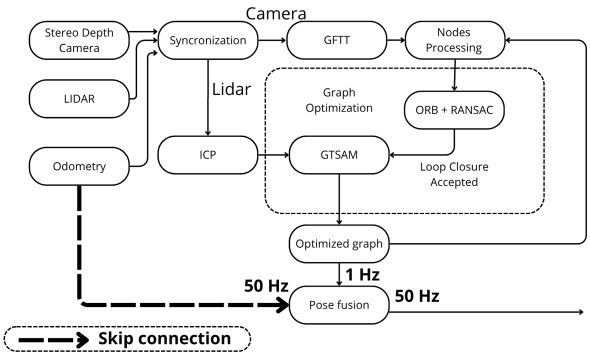

Fig. 1. Block diagram of Extended RTAB-Map architecture. The blue line indicates the skip connection from wheels and IMU odometry to pose fusion.

1) Odometry Calculation from Wheels and IMU: The most important step is to calculate the wheels and IMU odometry using wheel encoder speed and IMU data. The relationship can be expressed as follows:

$$\begin{bmatrix} v_x \\ v_y \\ v_\theta \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} \text{encoder\_to\_meter}\, \text{encoder\_speed}\, \cos\theta \\ \text{encoder\_to\_meter}\, \text{encoder\_speed}\, \sin\theta \\ \text{current\_IMUyaw} - \text{previous\_IMUyaw} \end{bmatrix} \quad (1)$$

Here, v_x represents the linear velocity of vehicle in the x-direction (m/s), v_y is the linear velocity of vehicle in the y-direction (m/s), and v_θ is the angular velocity of vehicle around the z-axis (rad/s). The term Δt denotes the time difference between the current and previous readings (20 ms). The variable encoder_to_meter is a conversion factor from encoder speed (in rpm) to meters per second. encoder_speed refers to the encoder output, θ is the yaw angle from the IMU in radians, while current_IMUyaw and previous_IMUyaw denote the consecutive IMU yaw readings in radians.

2) Utilize RTAB-Map Synchronization Strategy: To handle multiple sensors with multiple update rate too, we utilizing the RTAB-Map synchronization strategy. By default, RTAB-Map use exact sync strategy that require all the sensor update their data at a same time. That strategy difficult to implement because each sensor operates independently with difference update rate. So we change the synchronization strategy to approximate sync. This strategy allows each sensor to not have exactly same timestamp, but still close enough.

3) Utilize RTAB-Map Loop Closure Detection: The second optimization of RTAB-Map involves the use of the GFTT (Good Features to Track) algorithm as an image feature detector. As demonstrated in [7], GFTT provides a fast yet reliable feature extraction performance suitable for real-time applications. The third improvement applies the combination of ORB (Oriented FAST and Rotated BRIEF) and RANSAC (Random Sample Consensus) for feature matching. ORB provides a computationally efficient descriptor while maintaining robustness, and RANSAC filters out outliers from matched keypoints [8]. The output of this step is a loop-closure hypothesis, which indicates that the robot has revisited a previously mapped location. This hypothesis is used to refine and optimize the pose graph.

4) Utilize GTSAM Constraint: The fourth enhancement in RTAB-Map involves applying the ICP (Iterative Closest Point) algorithm to establish constraints between the nodes. ICP is utilized to compute the relative pose and covariance between two adjacent nodes [9]. This algorithm is selected because 2D LiDAR provide higher geometric accuracy compared to depth information from cameras [10].

The fifth stage involves full graph optimization using all nodes and edges. When a valid loop-closure hypothesis is detected, the corrected pose and covariance are calculated based on the ICP constraint. Those values used to optimize the entire pose graph through nonlinear optimization using the GTSAM library [11]. This optimization reduces accumulated drift and improves global pose estimation consistency.

5) Introduction of Skip Connection from Wheels and IMU to Pose Fusion: Despite these improvements, the standard RTAB-Map implementation yields an average pose update rate around 1 Hz when tested on 2.40 Ghz quadcore CPU with 16 GB RAM and without dedicated GPU. We introduce a skip connection from the wheels and IMU odometry to the pose fusion stage as shown in Fig. 1. The pose fusion employs a Kalman Filter to combine the corrected pose and covariance from RTAB-Map with the odometry-based pose and covariance from the wheels encoder and IMU [12]. The wheels and IMU odometry calculated by Eq. 1, serves as the velocity model in the Kalman Filter, while the RTAB-Map output acts as the position measurement model.

6) Fusion Strategy: The fusion strategy begins by assigning a low covariance value (around 0.01) to the wheels and IMU odometry, prioritizing responsiveness during the vehicle's initial motion. As the system detects more loop closures, the covariance of the RTAB-Map corrected pose gradually decreases, allowing it to dominate the fusion process. Consequently,

the final pose estimation becomes both accurate and smooth over time. The use of the Kalman Filter not only improves pose accuracy but also minimizes abrupt transitions during pose corrections. The resulting fused pose achieves a 50 Hz update rate on the same hardware setup, with centimeter-level precision suitable for real-time navigation.

B. Reduced Fast-SCNN

The Reduced Fast-SCNN is a lightweight semantic segmentation model designed for real-time lane detection in local navigation system. The Reduced Fast-SCNN model architecture is a modified version based on Fast-SCNN architecture [5] to optimize the lane detection inference time. The architecture is optimized by reducing the number of convolutional channels and replacing the pyramid pooling module with average pooling to enhance inference speed. The overall structure of the Reduced Fast-SCNN is illustrated in Fig. 2.

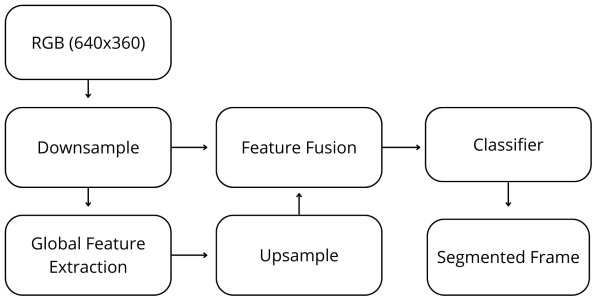

Fig. 2. Block diagram of the Reduced Fast-SCNN architecture.

1) Downsampling: The Reduced Fast-SCNN begins with a downsampling stage that reduces the input image (640×360×3) to a lower resolution of 21×11×32 (width × height × channels). While the original Fast-SCNN employs 64 channels after downsampling, our modified version uses 32 channels. This reduction effectively lowers the processing load, with a marginal trade-off in segmentation precision but still acceptable for static and structured industrial environments.

2) Global Feature Extraction: Following the same principle, the global feature extraction stage also reduces the number of channels. In the original architecture, this stage uses 128 channels, whereas in the Reduced Fast-SCNN it is limited to 64 channels. Additionally, the pyramid pooling module is replaced with a lightweight average pooling layer. Compared to pyramid pooling, average pooling simplifies the computation and improves inference speed, making it more suitable for CPU-only real-time operation.

3) Upsampling, Feature Fusion, and Classification: The upsampling, feature fusion, and classification stages retain the core structure of the original Fast-SCNN but operate with fewer convolutional channels. The final output of the Reduced Fast-SCNN is a binary segmentation mask representing two classes: lane and non-lane regions. This segmentation output is later utilized to compute a confidence level that influences the vehicle's navigation decisions, particularly in adjusting velocity control.

C. Integration of Both Systems

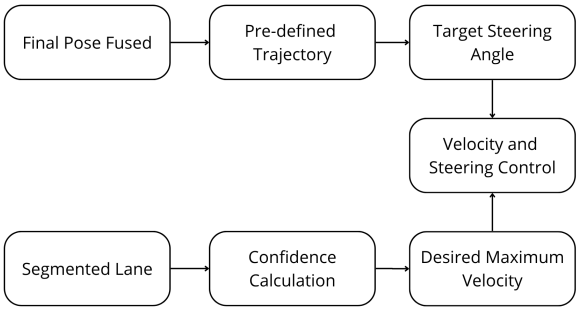

Fig. 3. Integration of the Extended RTAB-Map and Reduced Fast-SCNN for the navigation system.

Figure 3 illustrates the integrated navigation framework combining the Extended RTAB-Map for global localization and the Reduced Fast-SCNN for local perception. The navigation controller computes the desired heading angle to the next waypoint as:

$$\theta_{\text{direction}} = \tan^{-1}\left(\frac{y_{\text{wp}} - y_{\text{position}}}{x_{\text{wp}} - x_{\text{position}}}\right) - \theta_{\text{orientation}} \tag{2}$$

where $(x_{\text{wp}}, y_{\text{wp}})$ and $(x_{\text{position}}, y_{\text{position}})$ denote the waypoint and current vehicle positions, respectively, and $\theta_{\text{orientation}}$ is the current heading. Using the Bicycle Model [13], the target steering angle can be calculated using:

$$\delta_{\text{target}} = \tan^{-1}\left(\frac{2L\sin(\theta_{\text{direction}})}{D_{\text{lookahead}}}\right) \tag{3}$$

where L is the wheelbase and $D_{\text{lookahead}}$ the lookahead distance.

The confidence level from lane segmentation is defined as:

$$\text{normal} = \frac{\text{area of lane}}{\text{area of ROI}} \tag{4}$$

and used to modulate the target speed:

$$v_{\text{target\_max}} = v_{\text{max}} \cdot \text{normal}^2 \tag{5}$$

Where $v_{\text{target\_max}}$ is the dynamically maximum allowable speed. The v_{max} is the predefined maximum speed limit. The quadratic relationship ensures that speed decreases significantly in low-confidence scenarios, such as sharp turns or obstacles.

A first-order low-pass filter ensures smooth velocity transitions:

$$v_{\text{target}} = \alpha v_{\text{target\_max}} + (1-\alpha)v_{\text{target}} \tag{6}$$

where $\alpha = 0.055$. The resulting v_{target} and δ_{target} are passed to the low-level controller for real-time execution. This integrated approach achieves high-frequency localization and stable motion control on CPU-only hardware.

The θ_{target} of steering angle and v_{target} of velocity were sent to the low-level control layer to be executed in real time. By combining those two subsystems, the navigation system achieves real-time perception and high-frequency localization, enabling stable and accurate motion control even on limited hardware resources.

III. Results and Discussion

This section presents the experimental results obtained from the proposed navigation system. The evaluation focuses on three key aspects: (1) the accuracy of pose estimation with and without the Extended RTAB-Map, (2) the comparison of update rates between the default RTAB-Map and the Extended RTAB-Map, and (3) the inference performance comparison between the original Fast-SCNN and the proposed Reduced Fast-SCNN. All experiments were conducted on 2.40 Ghz quadcore CPU with 16 GB RAM and without dedicated GPU.

A. Wheels and IMU Odometry vs. Extended RTAB-Map

The first experiment compared the pose estimation accuracy between the wheels and IMU odometry approach and the Extended RTAB-Map. The quantitative results are shown in Table I.

TABLE I
Pose Estimation Result Using Wheels and IMU Odometry

Lap-	X (m)	Y (m)	Error (m)
1	0.28	0.39	0.48
2	0.30	0.43	0.52
3	0.24	0.49	0.55
4	0.18	0.32	0.37
5	0.37	0.41	0.55
Average Error			0.494

Table I is pose estimation result using wheels and IMU odometry after travelling for 500 meters and return to the zero position. Lap- in the Table I represents the lap number, while X (m) and Y (m) denote the final position of the vehicle (in meters) after traveling 500 meters. The Error (m) column shows the deviation of the estimated pose from the ground truth, which was defined as $(x = 0, y = 0)$. The average error of 0.494 m (49.4 cm) indicates that the wheels and IMU sensor produces significant drift over time, rendering it unsuitable for precise autonomous navigation. Therefore, pose estimation improvement using the Extended RTAB-Map is essential.

The results obtained using the Extended RTAB-Map are presented in Table II.

TABLE II
Pose Estimation Result Using Extended RTAB-Map

Lap-	X (m)	Y (m)	Error (m)
1	0.01	0.02	0.02
2	0.00	0.03	0.03
3	0.01	0.02	0.02
4	0.02	0.01	0.02
5	0.04	0.01	0.04
Average Error			0.026

The same parameters are used for Table II. After traveling 500 meters, the average pose error is around 0.026 m (2.6 cm), demonstrating a significant improvement over the wheels and IMU odometry approach. The result is sufficient for autonomous navigation in industrial environments, where centimeter-level accuracy is typically required.

Another effective way to visualize the improvement of pose estimation accuracy is by comparing the occupancy grid maps. The occupancy grid map reflects how well the estimated poses align spatially, providing a visual indicator of mapping consistency. Figure 4 presents the occupancy grid map produced using wheels and IMU odometry.

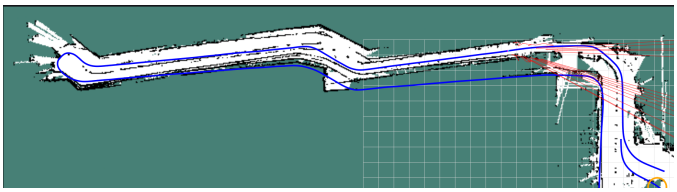

Fig. 4. Occupancy grid map generated using wheels and IMU odometry.

Figures 4 and 5 compare the results obtained from the wheels-IMU odometry and the proposed Extended RTAB-Map method. In both figures, the blue line represents the ground-truth trajectory used for evaluation.

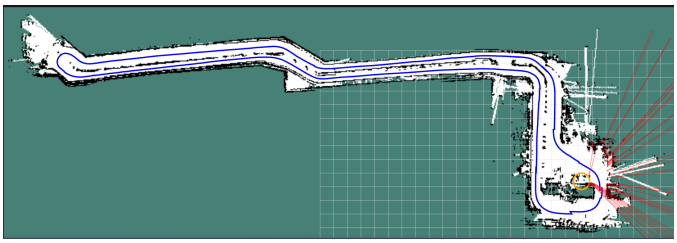

Fig. 5. Occupancy grid map generated using the Extended RTAB-Map method.

The difference between the two maps is clearly observable. The map generated using wheel and IMU odometry shows significant drift and deformation, leading to misaligned and distorted features. In contrast, the map produced by the Extended RTAB-Map aligns closely with the ground truth trajectory, exhibiting consistent geometry and improved global structure. This confirms the effectiveness of the proposed approach in enhancing pose estimation accuracy and consistency.

B. Update Rate: Default RTAB-Map vs. Extended RTAB-Map

The second experiment analyzed the update rate of the pose estimation between the default RTAB-Map and the Extended RTAB-Map. The measurement was conducted using the ROS topic statistics API, which computes the average publishing frequency of the pose topic. The results are shown in Table III.

TABLE III
UPDATE RATE COMPARISON BETWEEN DEFAULT AND EXTENDED RTAB-MAP

Lap-	RTAB-Map (Hz)	Extended RTAB-Map (Hz)
1	1.003	50.029
2	1.002	50.006
3	1.223	49.999
4	1.056	49.999
5	1.332	49.998
Average	1.123	50.006

Table III present the comparison of the average update rate between the default RTAB-Map and Extended RTAB-Map. It also present the update rate achieved by the proposed system. The average update rate increased from 1.123 Hz to 50.006 Hz. This demonstrates that by integrating the wheels and IMU odometry through the proposed skip connection and Kalman fusion significantly enhances temporal resolution, making real-time pose updates possible without requiring GPU acceleration.

C. Reduced Fast-SCNN Result

The third experiment evaluated the performance of the proposed Reduced Fast-SCNN model. A custom dataset of 797 labeled images with a resolution of 640×360×3 was developed specifically for this task. Data augmentation was applied using brightness and contrast adjustment ($p = 0.2$), shifting–scaling–rotation ($\pm 5\%$, $\pm 5\%$, $\pm 15°$, $p = 0.5$), and Gaussian blurring ($p = 0.1$) to simulate viewpoint and illumination variations. The model was trained using the Adam optimizer with a learning rate of 0.0001, a batch size of 4, and 100 epochs. A sample of the dataset is shown in Fig. 6.

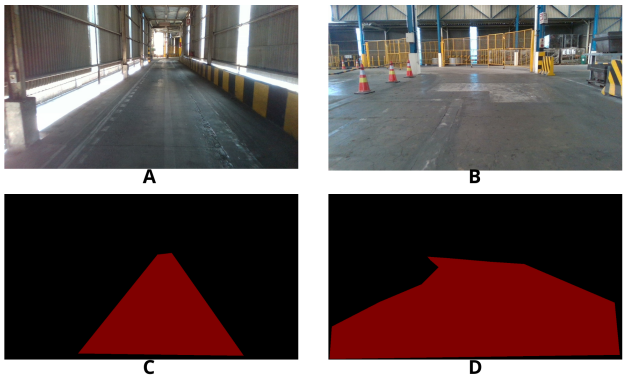

Fig. 6. The Research Dataset. A and B are the RGB images input. C and D are the mask images.

Figs. 6 shows RGB images input (image A and B) from the onboard camera, while images C and D show the corresponding segmentation masks. The segmentation results contain two classes: lane and non-lane regions. The lane pixels are marked in red, while the background is represented in black.

After training with 100 epochs and 0.5030 loss error, the model was converted into the ONNX format for deployment on CPU hardware. Inference was performed using the ONNX Runtime, which provided approximately a twofold increase in processing speed compared to the native PyTorch implementation [14]. Figure 7 shows preliminary results of lane detection using the Reduced Fast-SCNN.

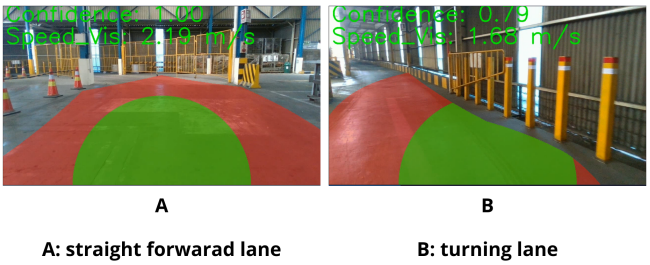

Fig. 7. Preliminary lane detection results using the Reduced Fast-SCNN. A is a full straight lane, while B is a turning lane.

In Fig. 7, the red mask indicates the detected lane area, while the green region represents the region of interest (ROI) used to calculate the confidence level. Image A shows a full straight lane where the ROI is completely filled, while Image B shows a turning condition with a truncated ROI, resulting in lower confidence.

The final inference results are displayed in Fig. 8.

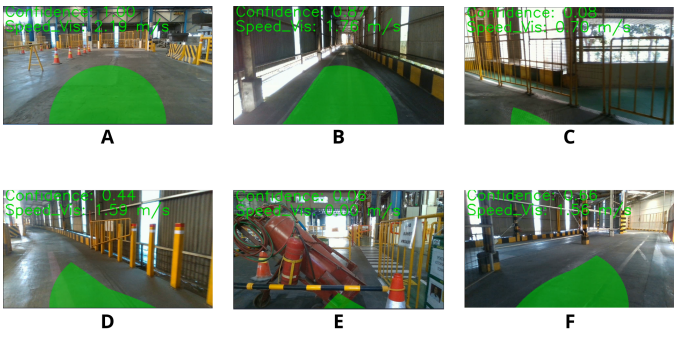

Fig. 8. Lane detection and velocity control results using the Reduced Fast-SCNN.

Figure 8 illustrates various driving scenarios. In Image A, the vehicle is in a full straight lane with an output velocity of 2.2 m/s. Image B depicts a narrow straight lane with a slightly reduced speed of 1.76 m/s. In Image C, the vehicle navigates a sharp turn while approaching a fence, resulting in a velocity of 0.70 m/s. Image D shows a left turn in a narrow area, where

the velocity is 1.59 m/s. In Image E, the vehicle encounters an obstacle directly in front, stopping completely at 0.00 m/s. Finally, Image F depicts a right turn in a narrow area with a speed of 1.56 m/s. Those results demonstrate the effectiveness of the Reduced Fast-SCNN in adjusting the vehicle's speed based on lane confidence and environmental conditions.

D. Fast-SCNN vs. Reduced Fast-SCNN Inference Time

The final experiment compared the inference time between the Fast-SCNN and the proposed Reduced Fast-SCNN models. Both were operated on the same hardware platform (2.40 Ghz quadcore CPU , 16 GB RAM, without dedicated GPU). The same test images with a resolution of 640×360×3 were used for consistency. The results are summarized in Table IV.

TABLE IV
COMPARISON OF INFERENCE TIME BETWEEN FAST-SCNN AND REDUCED FAST-SCNN

Frame Number	Fast-SCNN (ms)	Reduced Fast-SCNN (ms)
1	22.4	7.2
2	29.3	7.7
3	34.2	7.8
4	39.2	7.7
5	23.8	7.4
6	28.1	8.0
Average	29.5	7.63

Table IV shows that the Reduced Fast-SCNN achieves faster inference than the original Fast-SCNN, with average times of 7.63 ms and 29.5 ms per frame, respectively. This represents a 3.9× speedup, confirming the model's real-time capability on CPU-only hardware and its suitability for autonomous vehicles with limited computational resources.

IV. FUTURE DEVELOPMENT

There are two primary avenues to improve the local navigation system. The first is increasing robustness in lane detection. This can be achieved by incorporating temporal modeling like gated recurrent units (GRUs) [15] to exploit inter-frame dependencies and reduce flicker in the segmentation output. The second improvement is performing velocity and steering control using machine learning. In this approach, control commands can be calculated by leveraging the outputs of the lane detection module together with the fused pose estimation.

V. CONCLUSION

This study proposes a hybrid navigation system combining the Extended RTAB-Map for global navigation and the Reduced Fast-SCNN for local navigation system to improve autonomous vehicle performance. The Extended RTAB-Map enhances pose estimation by adding a skip connection from wheels and IMU odometry to a Kalman-based fusion stage, increasing update rate to 50 Hz and reducing error below 5 cm. The Reduced Fast-SCNN accelerates lane detection inference time with fewer convolutional channels and average pooling, achieving 7.63 ms inference on CPU. Integrated together, they enable accurate, real-time, and lightweight navigation suitable for CPU-only applications.

VI. ACKNOWLEDGMENT

This research was supported by Indonesia Smelting Technology (IST), Manufaktur Robot Industri (MRI), and ITS Robotics. The authors would like to thank the teams from IST and MRI for their assistance and collaboration, and the ITS Robotics team for their continued support. Finally, we extend our thanks to ChatGPT and GitHub Copilot for writing support during manuscript preparation.

REFERENCES

[1] R. Kazemi, R. Haidarimoghadam, M. Motamedzadeh, R. Golmohammadi, A. Soltanian, and M. R. Zoghipaydar, "Effects of shift work on cognitive performance, sleep quality, and sleepiness among petrochemical control room operators," *Journal of Circadian Rhythms*, vol. 14, p. 1, 2016. [Online]. Available: https://jcircadianrhythms.com/articles/10.5334/jcr.134

[2] D. Neven, B. D. Brabandere, S. Georgoulis, M. Proesmans, and L. V. Gool, "Towards end-to-end lane detection: an instance segmentation approach," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018, pp. 38–45, demonstrates real-time lane detection performance relevant to inference time analysis in autonomous vehicle navigation. [Online]. Available: https://arxiv.org/abs/1802.05591

[3] M. Labbé and F. Michaud, "Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," in *Journal of Field Robotics*, vol. 36, no. 2. Wiley Online Library, 2019, pp. 416–446.

[4] B. Clipp, J. Lim, J.-M. Frahm, and M. Pollefeys, "Parallel, real-time visual slam," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 3961–3968.

[5] A. C. Poudel, S. Liwicki, and R. Cipolla, "Fast-scnn: Fast semantic segmentation network," 2019.

[6] M. L. R. Lagahit and M. Matsuoka, "D-fast-scnn + combo loss: Improved road marking extraction on mobile lidar sparse point cloud-derived images," in *IGARSS 2023 - 2023 IEEE International Geoscience and Remote Sensing Symposium*, 2023, pp. 5684–5687.

[7] S. A. Khan Tareen and R. H. Raza, "Potential of sift, surf, kaze, akaze, orb, brisk, agast, and 7 more algorithms for matching extremely variant image pairs," in *2023 4th International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*, 2023, pp. 1–6.

[8] A. Vinay, A. S. Rao, V. S. Shekhar, A. C. Kumar, and K. N. B. Murthy, "Feature extraction using orb-ransac for face recognition," *Procedia Computer Science*, vol. 70, pp. 174–184, 2015, presented at 4th International Conference on Eco-friendly Computing and Communication Systems (ICECCS 2015).

[9] J. Zhang, Y. Yao, and B. Deng, "Fast and robust iterative closest point," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3450–3466, 2022.

[10] L. Hu, C. Wei, Y. Xu, Z. Zhang, L. Li, and X. Qian, "Fine registration of low-resolution infrared camera and low-beam lidar," in *2023 IEEE International Conference on Unmanned Systems (ICUS)*, 2023, pp. 925–930.

[11] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," *Georgia Institute of Technology*, 2012, available at https://gtsam.org.

[12] M. I. Zazuli, R. Dikairono, D. Purwanto, Muhtadin, and M. L. Hakim, "Sensor fusion system for localization of autonomous car," in *2024 International Conference on Green Energy, Computing and Sustainable Technology (GECOST)*, 2024, pp. 1–5.

[13] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Boston, MA: Springer, 2011.

[14] F. Xu, "Faster scalable ml model deployment using onnx and open source tools," in *2020 IEEE Infrastructure Conference*, 2020, pp. i–i.

[15] T.-W. Yu, M. A. Sarwar, Y.-A. Daraghmi, S.-H. Cheng, T.-U. İk, and Y.-L. Li, "Spatiotemporal activity semantics understanding based on foreground object segmentation: icounter scenario," *IEEE Access*, vol. 10, pp. 57 748–57 758, 2022.